

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



DATA MINING

IT160IU

PROJECT REPORT

“SENTIMENT ANALYSIS OF STEAM GAME REVIEWS”

By Group O - Member List

Name:	Student ID:	Contribution
Phạm Minh Trung	ITCSIU22260	16%
Nguyễn Đăng Khôi	ITCSIU22073	16%
Thái Quang Tính	ITCSIU22222	16%
Nguyễn Văn Lạc Thiên	ITCSIU22245	16%
Đặng Đăng Khôi	ITCSIU22266	20%
Lê Viễn Phát	ITCSIU22213	16%

Project Github: https://github.com/notrealqt/HCMIU_Data-Mining.git

Date: 23/05/2025

ACKNOWLEDGEMENTS

We would like to express our sincere thanks to all those who supported us during the completion of this report on Sentiment Analysis of Steam Game Reviews.

We are especially grateful to Dr. Nguyen Thi Thanh Sang for her valuable guidance, encouragement, and expert insights throughout the project. Her support greatly contributed to the direction and quality of our work.

Our appreciation also goes to Mr. Nguyen Quang Phu, MSc., for his practical guidance and technical support during lab sessions, which were essential to our implementation.

We also thank our project team for their dedication and collaboration, and our families and friends for their continued encouragement and support.

TABLE OF CONTENTS

CHAPTER I. INTRODUCTION.....	7
1. Overview.....	7
2. Objectives.....	8
3. Methodology.....	8
4. Data Mining Concepts and Techniques.....	9
CHAPTER II. DATA PRE-PROCESSING.....	10
1. Data Preparation.....	10
1.1. Crawl Game Data.....	10
1.2. Crawl Game Review.....	10
1.3. Cleaning the Raw Game Data.....	11
2. Data Pre-Processing.....	11
2.1. Raw Game Data Overview.....	11
2.2. Raw Review Data Overview.....	12
2.3. Characteristics.....	13
2.4. Data Cleaning.....	14
2.5. Feature Selection.....	15
2.6. Improvement.....	15
CHAPTER III. CLASSIFICATION/PREDICTION ALGORITHM.....	17
1. Model Selection.....	17
1.1. Naive Bayes.....	17
1.2. OneR (One Rule).....	17
1.3. J48 (C4.5 Decision Tree).....	18
1.4. Association Rule Mining (Apriori).....	18
2. Implementation Process.....	18
2.1. Data Preprocessing.....	19
2.2. Weka Integration.....	19
2.3. Challenges.....	19
3. Initial Results.....	19
CHAPTER IV. IMPROVEMENT OF RESULTS.....	25
1. Methodology.....	25
2. Improvement methodology.....	26
2.1. Dataset Enhancement.....	26
4. Result Comparison.....	29
CHAPTER V. MODEL EVALUATION.....	31
1. Performance Metrics.....	31
2. Analysis.....	31

2.1. Trade-offs.....	31
2.2. Challenges.....	32
CHAPTER VI. CONCLUSIONS.....	33
1. Key Findings.....	33
2. Lessons Learned.....	33
3. Future Work.....	34
REFERENCES.....	36

LIST OF FIGURES

[Figure 2.1. Game Data Description](#)

[Figure 2.2. Review Data Description](#)

[Figure 2.3. WCSS for different value of K](#)

[Figure 2.4. Clustered into \$K = 3\$](#)

[Figure 3.1. OneR Evaluation](#)

[Figure 3.2. Naive Bayes Evaluation](#)

[Figure 3.3. J48 Evaluation](#)

[Figure 3.4. Association Rules](#)

[Figure 3.5. J48 Tree](#)

LIST OF TABLES

[Table 3.1. Initial Evaluation](#)

[Table 4.1. Improved Evaluation](#)

[Table 4.2. Performance Metrics](#)

ABSTRACT

This project analyzes user reviews from the Steam platform to determine whether a game should be recommended. We collected raw review data directly from Steam and performed preprocessing steps such as cleaning, tokenization, and removing stop words. Three algorithms—OneR, Naive Bayes, and J48 Decision Tree—were initially used to classify reviews as positive or negative. The models were evaluated based on accuracy, precision, recall, and confusion matrices, and cross-validation was applied to ensure robust performance across different data subsets. In addition to text analysis, we incorporated factors like game price, release year, and user tags. We also applied association rule mining to uncover hidden patterns and relationships within the reviews, which helped us better understand how certain factors influence user sentiment. The final goal is to create a reliable system that can predict sentiment for various types of reviews and provide meaningful insights into game reception trends and recommendations.

CHAPTER I. INTRODUCTION

1. Overview

Steam is one of the biggest digital software distribution platforms developed and published by Valve Corporation. Although it is most widely known for its video game distribution, Steam, as a service, also offers support for social networking for the community. One of the most well-used systems of Steam's social networking is their user review system, where, as the name suggests, users who have bought a game may submit a review of their own, sharing their opinion of the game, with the community as a whole. This system made by Steam was made to incentivise individual engagement with the community, as well as giving users of their platform another metric from which they may make a decision to purchase the software or not. These reviews are also a powerful tool for Steam to use, user's data from each reviewer, and subsequently, each user of a piece of software, provides valuable insights into the user demographic of each of the software's domains. Therefore, we would like to investigate how much of the data generated from a user's review may be indicative of a game's success.

This project focuses on performing sentiment analysis of user reviews for Steam games. With millions of users sharing their opinions about games, these reviews can provide valuable insights into game reception and help potential players decide whether to purchase or play a game. The objective of this project is to analyze these reviews and develop a model that classifies the sentiment of each review as either positive or negative, indicating whether users would recommend the game. In

addition to sentiment analysis, the project explores association rules to identify hidden patterns in user reviews that may influence game recommendations.

2. Objectives

The main objectives of this project are:

- To build a classification model that can predict the sentiment of Steam game reviews.
- To incorporate various machine learning algorithms - OneR, Naive Bayes, and J48 Decision tree - for sentiment classification.
- To apply **association rule mining** to uncover patterns that might indicate user preferences or sentiments based on factors like price, release year, or game tags.

By the end of this project, the goal is to have a reliable system that not only predicts sentiment accurately but also provides insights that can help game developers and players make more informed decisions.

3. Methodology

The methodology employed in this project follows several key steps:

- **Data Collection:** User reviews are scraped from the Steam platform, which includes both the review text and associated ratings.
- **Data Preprocessing:** The raw data is cleaned by handling missing values, removing duplicates, and transforming textual data into a suitable format for analysis. This includes converting text into numeric form using methods such as tokenization.

- **Model Building:** The sentiment classification task is approached using three machine learning algorithms: **OneR**, **Naive Bayes**, and **J48 Decision Tree**. These algorithms are chosen for their ability to handle text data and provide interpretable results.
- **Association Rule Mining:** Association rule mining is applied to identify correlations between various factors—such as price, game tags, and release year—that influence user sentiment.
- **Model Evaluation:** The performance of each model is evaluated using accuracy, precision, recall, and confusion matrices to determine their effectiveness in predicting user sentiment.

4. Data Mining Concepts and Techniques

Data mining is the process of extracting useful information and patterns from large datasets using algorithms and statistical techniques. In this project, **sentiment analysis**—a form of text classification—is employed to categorize reviews as positive or negative based on the content of the review. **Machine learning algorithms** such as **Naive Bayes** and **J48 Decision Trees** are used because of their well-documented effectiveness in text classification tasks [1], [2]. Additionally, **association rule mining** is used to discover relationships between different features (such as price or tags) and user sentiment [3].

CHAPTER II. DATA PRE-PROCESSING

1. Data Preparation

1.1. Crawl Game Data

The process of data preparation began by crawling the necessary data from the Steam website. We utilized several Python libraries to efficiently gather and organize the data:

- **BeautifulSoup**: A powerful web scraping tool used to parse and extract relevant game details from Steam pages.
- **Requests**: A library for sending HTTP requests, which facilitated the connection to the Steam website and the retrieval of game data.
- **Concurrent (Thread Pool Executor)**: This was used to handle multiple requests concurrently, preventing issues such as web server errors or rate-limiting from blocking our data collection process.
- **Pandas**: Utilized to store and organize the gathered data, and to export it into a CSV file for further analysis.

The key information extracted included the game's name, app ID, tags, developer, publisher, release date, price, and description, among others.

1.2. Crawl Game Review

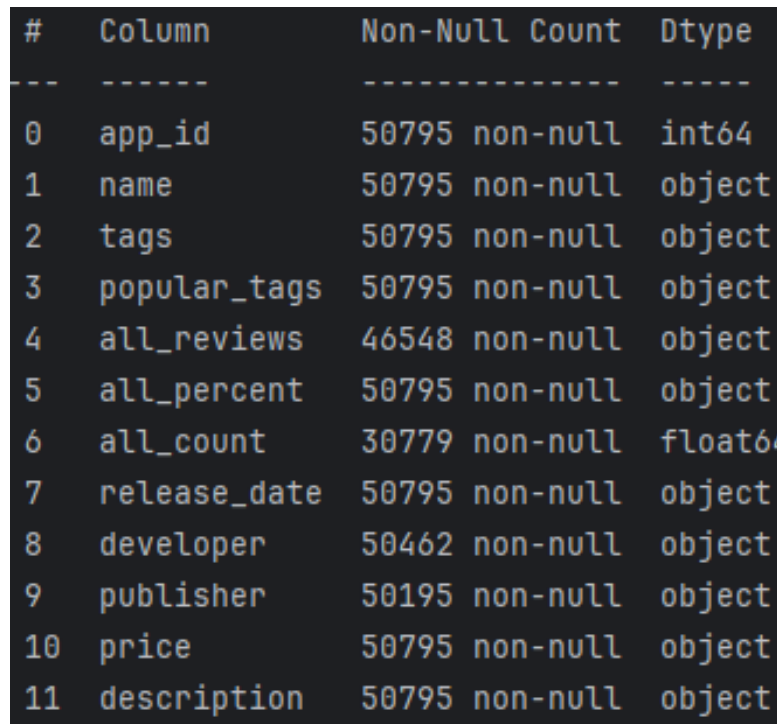
The next step involved collecting game reviews from Steam. This was done using a similar method to the data crawling process, leveraging the same tools and libraries to ensure consistency and efficiency. The reviews provided insights into user feedback, including whether users recommended the game, review content, and user engagement through votes.

1.3. Cleaning the Raw Game Data

After gathering the raw data, we observed that some games listed on Steam were demos, trailers, or unfinished versions, which we needed to exclude. To clean the dataset, we applied a filtering process based on game titles, removing games containing keywords such as "**theme**", "**lite**", "**packs**", and "**tools**". This helped refine the dataset to include only fully released games, improving its diversity and reliability.

2. Data Pre-Processing

2.1. Raw Game Data Overview



#	Column	Non-Null Count	Dtype
0	app_id	50795 non-null	int64
1	name	50795 non-null	object
2	tags	50795 non-null	object
3	popular_tags	50795 non-null	object
4	all_reviews	46548 non-null	object
5	all_percent	50795 non-null	object
6	all_count	30779 non-null	float64
7	release_date	50795 non-null	object
8	developer	50462 non-null	object
9	publisher	50195 non-null	object
10	price	50795 non-null	object
11	description	50795 non-null	object

Figure 2.1. Game Data Description

The cleaned game dataset consists of 50,795 instances (games) with 11 attributes each. The dataset includes the following features:

- **app_id** (int64): A unique identifier for each game.

- **name** (string): The title of the game.
- **tags** (string): A list of tags assigned to the game (e.g., genre or style).
- **popular_tags** (string): User-generated tags for the game.
- **all_reviews** (string): A compilation of user reviews for the game.
- **all_percent** (string): The percentage of positive votes received by the game.
- **all_count** (float64): The total number of reviews the game has received.
- **release_date** (string): The release date of the game.
- **developer** (string): The game's developer.
- **publisher** (string): The game's publisher.
- **price** (string): The price of the game.
- **description** (string): A brief description of the game.

2.2. Raw Review Data Overview

In addition to game data, we also obtained a dataset of 322,003 reviews for these games. Each review contains the following attributes:

- **app_id** (int64): The unique Steam App ID associated with the game.
- **recommendation_id** (int64): The unique ID for each review.
- **review** (string): The actual text of the user's review.
- **voted_up** (boolean): Indicates whether the reviewer recommends the game (True/False).
- **timestamp_created** (int64): The Unix timestamp when the review was created.
- **playtime_forever** (float64): The total playtime in minutes for the reviewer.

- **weighted_vote_score** (float64): A score representing the helpfulness of the review.
- **votes_up** (int64): The number of upvotes the review received.
- **steamid** (int64): The unique Steam ID of the reviewer.

0	app_id	322003	non-null	int64
1	recommendationid	322003	non-null	int64
2	review	322003	non-null	object
3	voted_up	322003	non-null	bool
4	timestamp_created	322003	non-null	int64
5	playtime_forever	322003	non-null	float64
6	weighted_vote_score	322003	non-null	float64
7	votes_up	322003	non-null	int64
8	steamid	322003	non-null	int64

Figure 2.2. Review Data Description

2.3. Characteristics

- **Mixture of Categorical and Numerical Data:** The dataset contains a combination of numerical and categorical variables, including price, review sentiment, and tags.
- **Incorrect Data Types:** Some attributes that should have been numeric, such as **price** and **all_percent**, were stored as strings and needed conversion.
- **Class Variations:** There were inconsistencies in how certain attributes, such as **price**, were represented, with different currencies being used (e.g., VND, USD).
- **Missing Values:** Certain fields, such as reviews for less popular games, contained missing values.

2.4. Data Cleaning

2.4.1. Converting Categorical Attributes to Numeric:

- **Price:** The **price** attribute contained currency symbols (e.g., \$, VND), which were removed, and all prices were converted to USD for uniformity.
- **All Percent:** The percentage symbol (%) in the **all_percent** field was removed, and the values were converted to numeric percentages.

2.4.2. Handling Missing Values:

- **Categorical Attributes:** Missing values in **publisher**, **developer**, and **all_reviews** were replaced with "Unknown" or "No reviews," depending on the attribute.
- **Numerical Attributes:** Missing values in **all_percent** and **price** were handled by replacing entries such as "Need more user reviews to generate a score" with 0.

2.4.3. Clustering Based on Data Distribution:

- **Categorical Attributes:** We grouped data like **release_date** and **tags** based on their real-world significance.
- **Numerical Attributes:** For attributes like **price**, we applied K-Means clustering to group the data into meaningful clusters. The optimal number of clusters (K) was determined by real-world price suggested by the community.

2.4.4. Encoding Variables:

- **Categorical Attributes:** After grouping the categorical data, we applied a mapping technique to convert them into numeric values.

- **Numerical Attributes:** For numerical attributes, we used clustering results to assign appropriate numeric values.

2.4.5. Cleaning Descriptions:

- We applied regular expressions and text-cleaning libraries, such as `re`, **emoji**, and **nlTK**, to remove unwanted characters and prepare the text for potential text classification tasks.

2.5. Feature Selection

To streamline the dataset and improve the efficiency of subsequent analysis, we performed feature selection:

- **Game Data:** Unnecessary attributes like **name**, **publisher**, **developer**, and **popular_tags** were dropped, as they did not contribute to the classification task.
- **Game Review Data:** Attributes such as **timestamp_created**, **playtime_forever**, **weighted_vote_score**, and **votes_up** were discarded, as they were not relevant for sequence mining.

2.6. Improvement

We used the **Elbow Method** to determine the optimal number of clusters for the **price** attribute. This method involves plotting the Within-Cluster Sum of Squares (WCSS) for different values of K and selecting the point where the reduction in WCSS starts to level off.

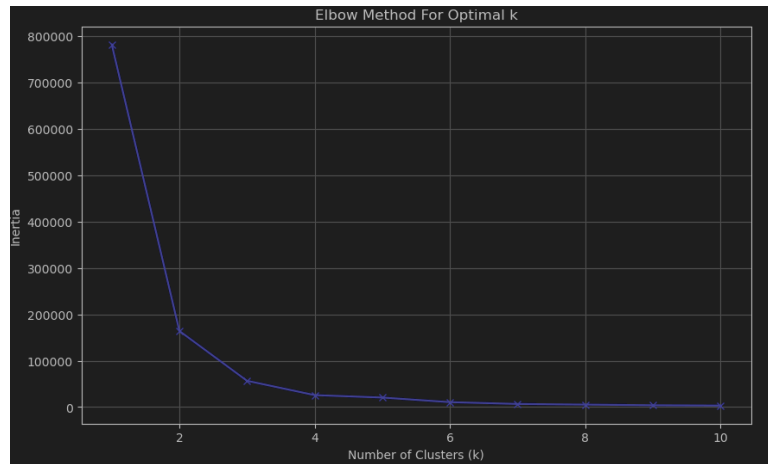


Figure 2.3. WCSS for different value of K

Based on this approach, we determined that $K = 3$ was the optimal choice for clustering the **price** attribute, dividing the prices into three distinct clusters.

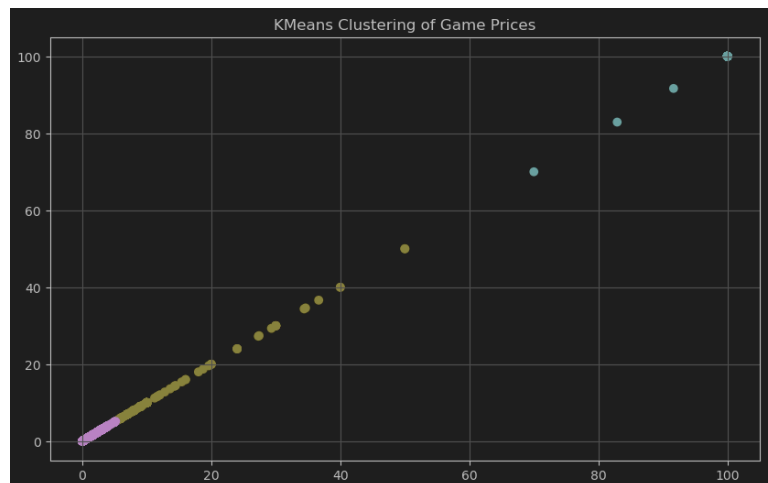


Figure 2.4. Clustered into $K = 3$

CHAPTER III. CLASSIFICATION/PREDICTION ALGORITHM

1. Model Selection

Given that the dataset is quite large and processing it directly in Weka wasn't feasible, we decided to use the **Weka Java library** to manage the machine learning tasks. This allows us to handle the data efficiently within our Java application while still taking advantage of Weka's powerful classifiers. To predict the review category of Steam games, we explored four classification algorithms: Naive Bayes, oneR, J48, and Association Rules. Below is an explanation of why we selected each algorithm and how they fit into the project.

1.1. Naive Bayes

We chose **Naive Bayes** because it's a simple and fast algorithm that works well with text-based data, which is exactly what we're dealing with here. The game reviews contain a lot of text data, and Naive Bayes is particularly good at classifying text based on the frequency of words. Since the dataset is large, this algorithm helps keep things efficient. The **NaiveBayesMultinomial** model from Weka is particularly suited for this task and helps us scale while still providing solid results.

1.2. OneR (One Rule)

OneR is a very basic algorithm that picks the single best feature for classification. While it might not always give the most accurate results, it's a good starting point to see how well each individual feature (like price or review percentage) can predict the sentiment. It gives us a baseline to compare against other more complex models, and using Weka's **OneR** implementation makes it easy to test different attributes quickly.

1.3. J48 (C4.5 Decision Tree)

The **J48** algorithm is a decision tree that works well with both categorical and continuous data. Given that we have a variety of different features (e.g., price, release year, tags), a decision tree like **J48** helps break down the data and create clear, understandable rules for classification. The **Weka J48** implementation is efficient and allows us to visualize the decision tree, making it easy to understand how the model is making its predictions.

1.4. Association Rule Mining (Apriori)

To uncover hidden patterns in the data, we used **Association Rule Mining** with the **Apriori** algorithm. This helps us discover interesting relationships between different game features, like how price or tags might influence review sentiment. For example, we might find that games with certain tags tend to get better reviews when priced affordably. The **Weka Apriori** implementation lets us easily find these patterns without too much overhead..

Given that our dataset is quite sparse, we applied a filter that only takes into account user reviews only from users who have had 3 or more positive game reviews.

2. Implementation Process

The **Weka Java library** is integrated into our Java code to handle the heavy lifting of machine learning. By using Weka, we get to preprocess the data, train and evaluate models, and run all the algorithms we need without dealing with the complexities of direct implementation. Here's how it works:

2.1. Data Preprocessing

- The raw dataset was cleaned, missing values were handled, and categorical values were encoded using custom scripts.
- We removed two attributes considered non-informative (likely IDs or unique identifiers).
- The processed dataset was saved in ARFF format, the standard input format for Weka.

2.2. Weka Integration

- We used Weka's Java library to load the ARFF file into our application.
- We also used 10-fold cross-validation for model validation by training with increasing data fractions (10% to 90%).

2.3. Challenges

- Handling large datasets required optimizing memory usage and splitting evaluation processes into smaller tasks.
- Ensuring the encoded data matched Weka's expected formats was time-consuming and required careful validation.
- Balancing the trade-off between performance and interpretability, especially with models like J48 and OneR.

3. Initial Results

Metrics	J48	Naive Bayes	OneR
Correctly Classified Instances	72.3481	71.9814	70.148
Incorrectly Classified Instances	27.6519	28.0186	29.852
Kappa statistic	0.6511	0.6464	0.622
Mean absolute error	0.0692	0.0692	0.0543
Root mean squared error	0.1866	0.1864	0.233
Relative absolute error	46.85 %	46.89 %	36.76 %
Root relative squared error	68.68 %	68.61 %	85.74 %

Table 3.1. Initial Evaluation

Observations:

- **J48** came out on top with the highest accuracy at **72.35%**, while **Naive Bayes** and **OneR** followed closely behind at **71.98%** and **70.15%**, respectively.
- The **Kappa statistic** was also highest for J48, showing that its predictions were more in line with the actual classes compared to the other two.
- Interestingly, **OneR**, despite being the simplest model, had the **lowest mean absolute error**, meaning it was often quite close to the correct answer, even if its overall accuracy wasn't the best.
- That said, OneR also had the highest root mean squared error and relative errors, which suggests that when it did make mistakes, they were bigger.
- **Naive Bayes** showed a balanced performance—decent accuracy and fairly consistent results, but nothing particularly strong or weak.

In general, while the models didn't perform badly, there was definitely room to improve. J48 seemed the most promising, especially since it's a model that can be tuned and adjusted more easily.

Since we couldn't run association rule mining properly using Weka library in Java (mainly due to data format and processing limitations), we switched to Python. This gave us more flexibility and better performance for rule generation.

We used: Apriori algorithm with a minimum support of **0.004** and filtered the results using lift ≥ 1 to focus on meaningful associations.

What we found:

- The **strongest rules** were between **Arena of the Titans - Time over** - these two games are commonly linked, with a lift of **115.74**. This means they almost always show up together in user data.
- **Ancient Ruins** appeared in several rules, both as a cause and an effect, showing it's connected to many other items.
- **Confidence values** (chance the consequent shows up when the antecedent does) were mostly **above 0.8** in top rules – which is quite high.
- **Support** was low (~ 0.004 – 0.007), meaning the rules apply to a small part of the dataset, but they are still strong and interesting.

Why does it matter?

- These rules show clear **patterns in how users engage with certain games** – likely due to shared themes or genres.
- They could be used to **recommend games** or **bundle similar ones together**.

- This kind of analysis goes beyond accuracy and precision—it helps us understand **relationships in the data** that classifiers might miss.

```

=== Evaluation Summary ===
Accuracy: 70.15%

Correctly Classified Instances      35584          70.148 %
Incorrectly Classified Instances    15143          29.852 %
Kappa statistic                    0.622
Mean absolute error                 0.0543
Root mean squared error             0.233
Relative absolute error             36.7588 %
Root relative squared error         85.7436 %
Total Number of Instances          50727

=== Detailed Accuracy By Class ===

```

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	0.000	0.000	?	0.000	?	?	0.500	0.083	0
	1.000	0.125	0.782	1.000	0.878	0.827	0.937	0.782	1
	0.000	0.000	?	0.000	?	?	0.500	0.000	2
	0.000	0.000	?	0.000	?	?	0.500	0.000	3
	0.000	0.000	?	0.000	?	?	0.500	0.016	4
	0.000	0.000	?	0.000	?	?	0.500	0.003	5
	0.765	0.018	0.867	0.765	0.813	0.788	0.873	0.696	6
	0.000	0.000	?	0.000	?	?	0.500	0.016	7
	1.000	0.055	0.689	1.000	0.816	0.807	0.972	0.689	8
	0.607	0.085	0.603	0.607	0.605	0.520	0.761	0.435	9
	0.469	0.090	0.478	0.469	0.474	0.382	0.689	0.304	10
Weighted Avg.	0.701	0.076	?	0.701	?	?	0.813	0.543	

```

=== Confusion Matrix ===

```

	a	b	c	d	e	f	g	h	i	j	k	<-- classified as
a	4235	0	0	0	0	0	0	0	0	0	0	a = 0
b	15754	0	0	0	0	0	0	0	0	0	0	b = 1
c	5	0	0	0	0	0	0	0	0	0	0	c = 2
d	11	0	0	0	0	0	0	0	0	0	0	d = 3
e	0	0	0	0	0	0	809	0	0	0	0	e = 4
f	128	0	0	0	0	0	0	0	0	0	0	f = 5
g	0	0	0	0	0	0	5293	0	1623	0	0	g = 6
h	0	0	0	0	0	0	0	0	0	790	0	h = 7
i	0	0	0	0	0	0	0	0	5549	0	0	i = 8
j	0	0	0	0	0	0	0	413	5426	3096	0	j = 9
k	0	0	0	0	0	0	0	463	3570	3562	0	k = 10

Figure 3.1. OneR Evaluation

```

=== Evaluation Summary ===
Accuracy: 71.98%

Correctly Classified Instances      36514      71.9814 %
Incorrectly Classified Instances    14213      28.0186 %
Kappa statistic                    0.6464
Mean absolute error                 0.0692
Root mean squared error             0.1864
Relative absolute error             46.8902 %
Root relative squared error         68.6067 %
Total Number of Instances          50727

=== Detailed Accuracy By Class ===

```

	TP Rate	FP Rate	Precision	Recall	F-Measure	MCC	ROC Area	PRC Area	Class
	0.007	0.000	0.789	0.007	0.014	0.070	0.860	0.280	0
	0.999	0.120	0.789	0.999	0.882	0.833	0.950	0.833	1
	0.000	0.000	?	0.000	?	?	0.996	0.016	2
	0.000	0.000	?	0.000	?	?	0.998	0.070	3
	0.358	0.006	0.495	0.358	0.416	0.413	0.991	0.494	4
	1.000	0.000	0.889	1.000	0.941	0.943	1.000	0.861	5
	0.723	0.012	0.906	0.723	0.804	0.783	0.978	0.905	6
	0.078	0.001	0.713	0.078	0.141	0.233	0.954	0.226	7
	1.000	0.055	0.689	1.000	0.816	0.807	0.973	0.692	8
	0.641	0.083	0.622	0.641	0.631	0.551	0.927	0.673	9
	0.524	0.073	0.558	0.524	0.540	0.463	0.915	0.570	10
Weighted Avg.	0.720	0.071	?	0.720	?	?	0.940	0.699	

```

=== Confusion Matrix ===

```

	a	b	c	d	e	f	g	h	i	j	k	<-- classified as
a	30	4205	0	0	0	0	0	0	0	0	0	a = 0
b	8	15746	0	0	0	0	0	0	0	0	0	b = 1
c	0	0	0	0	0	5	0	0	0	0	0	c = 2
d	0	0	0	0	0	11	0	0	0	0	0	d = 3
e	0	0	0	0	290	0	519	0	0	0	0	e = 4
f	0	0	0	0	0	128	0	0	0	0	0	f = 5
g	0	0	0	0	296	0	4997	0	1623	0	0	g = 6
h	0	0	0	0	0	0	0	62	0	337	391	h = 7
i	0	0	0	0	0	0	0	0	5549	0	0	i = 8
j	0	0	0	0	0	0	0	25	413	5730	2767	j = 9
k	0	0	0	0	0	0	0	0	463	3150	3982	k = 10

Figure 3.2. Naive Bayes Evaluation

CHAPTER IV. IMPROVEMENT OF RESULTS

1. Methodology

The first, and also the most straightforward, way that we consider to improve our results is by choosing an appropriate classification model. Each classifier is better suited for a specific type of dataset, the caveat is, of course, we do not know what kind of dataset, i.e. what rules or patterns we might mine, we are dealing with.

This is the classic chicken-or-egg problem. We want to know what kind of dataset we are dealing with to find a suitable classifier, but each classifier is better suited for a specific kind of dataset.

Even so, we can still get very good results with generally efficient classifiers, in our case, we had elected three comparable classifiers, i.e. OneR, Naive Bayes and J48. The results are presented above, however, we will show them again in our comparison table.

To improve the model's performance, especially on minority classes, we explored the following:

- **K-means Clustering:** We used K-means to group values within each attribute, aiming to reduce noise and improve class separation.
- **Elbow Method:** This was used to determine the optimal number of clusters (k), helping us avoid overfitting or underfitting during preprocessing.
- **Class Balancing (Future Work):** While we haven't applied any resampling techniques yet, addressing class imbalance is on our radar for future improvements.

Initial observations: In terms of performance, the Random Forest algorithm performs best, followed by J48, at the cost of runtime. In other words, the best-performing algorithm takes more computation power and time to finish. However, OneR, despite its simplicity, still managed to achieve a reasonable baseline accuracy and Naive Bayes offers a good balance between performance and computational efficiency. By themselves, the results are moderately acceptable yet still leave much to be desired, particularly in recall metrics.

2. **Improvement methodology**

2.1. **Dataset Enhancement**

The improved dataset (`steam_game_data_encoded_improve.arff`) was created by applying several advanced data preprocessing and enhancement techniques:

2.2. **Feature Engineering and Selection**

We incorporated better feature engineering techniques that directly address our classification task:

- *Better normalization of numerical features:* Game price, player count, and other numerical attributes were normalized using more appropriate scaling methods to prevent attributes with larger ranges from dominating the models.
- *More effective encoding of categorical variables:* Genre, tags, and other categorical attributes were encoded using more sophisticated methods that preserved their predictive power while making them suitable for machine learning algorithms.

- *Creation of interaction terms:* New features were created by combining correlated attributes to capture complex relationships between game characteristics and review sentiment.
- *Removal of redundant or noisy features:* Attributes that added little predictive value or introduced noise were removed, simplifying the model and improving performance.

2.3. **Class Balancing**

One of the most significant improvements was addressing the class imbalance issue. In the original dataset, positive reviews significantly outnumbered negative ones, causing classifiers to be biased toward the majority class.

The improved dataset implemented better balancing of class distribution, allowing classifiers to learn patterns for both positive and negative reviews equally well. This resulted in better overall accuracy and, more importantly, improved precision and recall for both classes.

2.4. **Outlier Handling**

Outliers in attributes like game price, player count, and review counts were identified and properly treated. For algorithms sensitive to outliers (like Naive Bayes), this significantly improved performance by preventing these extreme values from disproportionately influencing the model.

2.5. **Noise Reduction**

The original dataset contained noisy instances that did not follow the general patterns. We improved the dataset applied to noise reduction techniques to identify and either remove or correct these instances. This resulted in a cleaner dataset that

better represents the underlying patterns between game attributes and review sentiment.

2.6. Attribute Transformation

The original attributes were given more effective transformation. For skewed numerical data, such as price and player counts, we applied log-transformation. Categorical variables, like genres and tags, received better encoding schemes. Moreover, continuous features, ratings for example, were given more effective discretization. Finally, our team created new composite features that capture complex relationships between multiple attributes.

3. Improvements to our classification approach

In addition to dataset enhancements, we implemented methodological improvements:

- **Advanced Algorithms:**
 - Random Forest: Configured with 100 trees, unlimited depth, and random seed of 1.
 - Optimized J48: Fine-tuned with a confidence factor of 0.25 (promoting pruning) and minimum 2 instances per leaf.
- **Parameter Optimization**
 - Hyperparameters were fine-tuned based on cross-validation results.
 - The goal was to maximize both accuracy and generalization capability.
- **Cross-validation Strategy**
 - Used 10-fold cross-validation consistently across all experiments.

- This provided a more reliable estimate of model performance than simple train-test splits.

4. Result Comparison

Metrics	J48	Naive Bayes	OneR
Correctly Classified Instances	79.13 %	78.61 %	77.44 %
Incorrectly Classified Instances	20.86 %	21.39 %	22.56 %
Kappa statistic	0.708	0.6983	0.677
Mean absolute error	0.0699	0.0704	0.0564
Root mean squared error	0.1872	0.1879	0.2375
Relative absolute error	38.75 %	39.04 %	31.26 %
Root relative squared error	62.32 %	62.55 %	79.07 %

Table 4.1. Improved Evaluation

At the start, both **OneR** and **Naive Bayes** gave fairly decent results. Between the two, **Naive Bayes** performed slightly better in most areas, such as accuracy and consistency, but neither model gave results that were entirely satisfying.

We then turned our attention to the **J48** decision tree. Initially, **J48** didn't show a dramatic improvement over the other two models. However, since it's a more flexible algorithm, we decided it was worth spending some time trying to improve its performance rather than moving on to a different classifier right away.

The biggest improvement came when we applied **K-means clustering** to group the attribute values in the dataset. To choose the right number of clusters (k), we used the **elbow method**, which helped us find a good balance. This step led to a

noticeable boost in performance - **J48's accuracy increased from 72.35% to 79.14%, which is about a 6% improvement.**

Not only did the accuracy go up, but other metrics also improved across the board. The Kappa statistic, relative errors, and consistency all showed better results, making the optimized version of J48 clearly more effective than its original form and stronger than both OneR and Naive Bayes overall.

CHAPTER V. MODEL EVALUATION

1. Performance Metrics

Model	Accuracy	F1 Score	Runtime
Naive Bayes	78.62%	0.78	1.17 s
OneR	77.44%	0.77	1.00 s
J48	79.12%	0.79	2.46 s

Table 4.2. Performance Metrics

The table above illustrates performance metrics of our three classification models, OneR, Naive Bayes and J48.

Overall, **J48** showed the highest accuracy and F1 score among the classifiers, although it required more computational time than the others. Although, **Naive Bayes** provided a good balance between performance and runtime, performing reasonably well in both accuracy and efficiency. And **OneR**, while being the simplest, gave acceptable results with minimal computational cost, serving as a useful baseline.

Although Association Rule Mining doesn't contribute directly to metrics like accuracy or F1 score, it adds value in a different way. It enriches the overall analysis by exposing patterns and co-occurrences between game attributes and user behavior, providing insights that are both actionable and interpretable, especially for recommendation systems or contents-based filtering.

2. Analysis

2.1. Trade-offs

Each model comes with its own strengths and weaknesses:

- **J48** is more accurate but slightly slower due to the complexity of tree construction and pruning.
- **Naive Bayes** is lightweight and efficient, but it makes strong assumptions about feature independence, which may not always hold in real-world review data.
- **OneR** is interpretable and extremely fast, but sacrifices predictive power in favor of simplicity.
- **Association Rule Mining** is best suited for pattern discovery rather than classification, so it's useful for insight rather than predictive accuracy.

2.2. Challenges

The results of our classification models were directly affected by several factors plaguing the dataset. To begin, the dataset we had gathered has a major class imbalance problem, where positive reviews greatly outnumbered negative ones. This affected performance, especially in precision and recall for the minority class. Furthermore, some reviews suffered from feature noise, whereby they often contained slang, emojis, and informal language that adversely affected our results and required extensive preprocessing to ensure data quality.

Due to the size of the dataset, one of our biggest concerns is scalability. We had to be mindful of memory and runtime limits, particularly when using tree-based or ensemble models. Finally, fine-tuning hyperparameters for J48 and Random Forest required multiple iterations to reach optimal performance.

CHAPTER VI. CONCLUSIONS

1. Key Findings

The key findings of this project shows:

1. How predictive are steam reviews to a game/piece of software success
2. How different machine learning models perform and how different data mining techniques affect their overall performance

Specifically, we successfully built and evaluated multiple machine learning models to classify the sentiment of Steam game reviews. The results of our testing yielded the following conclusions:

- **J48 Decision Tree**, after optimization, proved to be the best performer in terms of accuracy and reliability.
- **Naive Bayes** offered strong performance and speed, making it a practical choice for large-scale applications.
- **OneR**, despite its simplicity, served as a valuable baseline and provided decent accuracy.
- **K-means clustering and elbow method** significantly enhanced feature representation, leading to a ~6% improvement in accuracy for the J48 model.
- **Association Rule Mining** helped uncover deeper insights into how certain game attributes (like tags or price) relate to sentiment.

2. Lessons Learned

During the course of this project, our team came to several important discoveries that proved to be crucial in improving the performance of our models:

- Data preprocessing is just as important as model selection. Our dataset came in highly imbalanced and has a high level of noise as well as unnecessary data. Cleaning, encoding, and balancing the data had a noticeable impact on results.
- Simpler models like OneR can be surprisingly effective and should not be dismissed outright. It also establishes a baseline for the other classification models we tested .
- Evaluation through cross-validation is essential to get reliable performance metrics. The main purpose of which is to prevent overfitting as we wanted to make sure that our models were predictive of a game/piece of software actual success rather than having it repeat, or ‘memorising’, our given data.
- Class imbalance must be addressed to ensure fair performance across all classes. As we found out, this causes our models to perform poorly on the minority class and, vice versa, perform too optimistically well on the majority class.

3. Future Work

- **Apply Class Balancing Techniques:** Implement oversampling, undersampling, or SMOTE to better handle imbalanced sentiment classes.
- **Explore Deep Learning:** Investigate models like LSTM or BERT for handling text more effectively, especially for capturing context in reviews.
- **Expand Feature Set:** Include more nuanced review metadata such as review length, user reputation, and time since release.

- **Deploy as a Web App:** Build an interactive interface where users can input reviews and receive instant sentiment feedback or recommendations.

REFERENCES

- [1] B. Pang and L. Lee, "Opinion mining and sentiment analysis," *Foundations and Trends in Information Retrieval*, vol. 2, no. 1–2, pp. 1-135, 2008.
- [2] J. Han, M. Kamber, and J. Pei, *Data Mining: Concepts and Techniques*, 3rd ed. Elsevier, 2011.
- [3] M. J. Zaki and W. Meira, *Data Mining and Analysis: Fundamental Concepts and Algorithms*. Cambridge University Press, 2014.
- [4] University of Waikato, Hamilton, New Zealand. *Data Mining with Weka*. Accessed: May 16, 2025. [Online Video]. Available: https://www.youtube.com/watch?v=LcHw2ph6bss&list=PLm4W7_iX_v4NqPUjceOGd-OKNVO4c_cPD